IMEN891K / IDSC501 / AXGS-501 · DATA ENGINEERING · FALL 2026

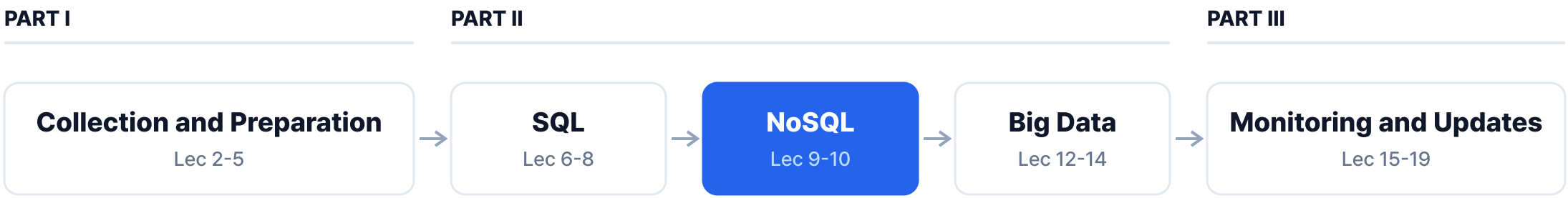
10.

NoSQL II: Document Stores

Sangwoo Mo · POSTECH

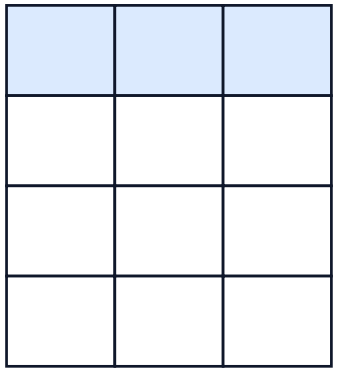
Industrial & Management Engineering

Where We Are


PART I
PART II
PART III
Collection and Preparation
Lec 2-5
SQL
Lec 6-8
NoSQL
Lec 9-10
Big Data
Lec 12-14
Monitoring and Updates
Lec 15-19

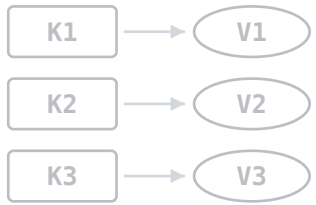
Recap: Types of NoSQL

SQL

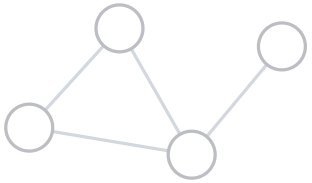


Relational

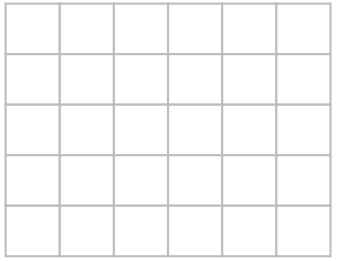
NoSQL



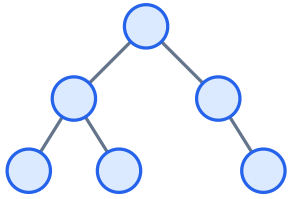
Key-value



Graph



Wide-column



Document

Last week: four data models. Today: the **document** model, in MongoDB

NoSQL Databases

- We talked about NoSQL data models
- We will now study one implementation in depth
    - MongoDB
        - BSON document database
        - Related document models appear in Couchbase, DynamoDB, and Firestore

Outline: NoSQL II

01	**Documents and Queries**	describe documents and ask for data
02	**Finding Documents**	find the records and fields you need
03	**Summarizing Documents**	turn documents into useful summaries
04	**Updating Documents**	change documents and check the result

MongoDB Data Model

Collections and documents, in relational terms

MongoDB	Relational DBMS
Database	Database
Collection	Relation
Document	Row/Record
Field	Column

Same nouns, one difference: documents in a collection **need not share a schema**

MongoDB Data Model

What a field value can be

Document = `{…, field: value, …}`

- Where value can be:
  - Atomic
  - A document
  - An array of atomic values
  - An array of documents

```
{ qty: 1,  status: "D",  size: { h: 14, w: 21 },  tags: ["a", "b"] }
```

Can also mix and match, e.g., array of atomics and documents, or array of arrays [Same as the JSON data model]

MongoDB Data Model

The `_id` field

Special field in each document: `_id`

- Primary key
- Will also be indexed by default
- If it is not present during ingest, it will be added
- Will be first attribute of each doc.
- This field requires special treatment during projections as we will see later

MongoDB Query Language (MQL)

Collections in, collections out

- Input = collections, output = collections

MongoDB Query Language (MQL)

Three kinds of query

- Input = collections, output = collections
- Three main types of queries in the query language
  - Retrieval: Restricted SELECT-WHERE-ORDER BY-LIMIT type queries
  - Aggregation: A bit of a misnomer; a general pipeline of operators
    - Can capture Retrieval as a special case
    - But worth understanding Retrieval queries first...
  - Updates

MongoDB Query Language (MQL)

One collection per call chain

- Input = collections, output = collections
- Three main types of queries in the query language
  - Retrieval: Restricted SELECT-WHERE-ORDER BY-LIMIT type queries
  - Aggregation: A bit of a misnomer; a general pipeline of operators
    - Can capture Retrieval as a special case
    - But worth understanding Retrieval queries first…
  - Updates
- All queries are invoked as
  - `db.collection.operation1(…).operation2(…)…`
    - collection: name of collection
  - Unlike SQL which lists many tables in a FROM clause, MQL is centered around manipulating a single collection

Some MQL Principles: Dot (.) Notation

- `"."` is used to drill deeper into nested docs/arrays
- Recall: a value is atomic, a nested document, or an array of either
- Examples:
  - `"instock.qty"` → qty field within the instock field
    - Only when instock is a nested doc or an array of them
      - Nested doc: its qty field
      - Array: the qty field of any document in it
  - `"instock.1"` → second element within the instock array
    - An atomic value or a nested document
  - `"instock.1.qty"` → qty field of the second document in the instock array

Note: such dot expressions need to be in quotes

Some MQL Principles: Dollar ($) Notation

Operators in the key position

- `$` indicates that the string is a special keyword
  - E.g., `$gt`, `$lte`, `$add`, `$elemMatch`, ...
- Used as the “field” part of a “field : value” expression
- If binary operator, it is usually written as:
  - `{LOperand : { $keyword : ROperand }}`
  - e.g., `{qty : {$gt : 30}}`

```
{qty : {$gt : 30}}
```
→ get values greater than 30

Some MQL Principles: Dollar ($) Notation

Operators over an argument list

- `$` indicates that the string is a special keyword
  - E.g., `$gt`, `$lte`, `$add`, `$elemMatch`, ...
- Used as the “field” part of a “field : value” expression
- If binary operator, it is usually written as:
  - `{LOperand : { $keyword : ROperand }}`
  - e.g., `{qty : {$gt : 30}}`
- Alternative: arrays
  - `{$keyword : [argument list]}`
  - e.g., `{$add : [1, 2]}`

`{$add : [1, 2]}`
→ add the elements in the list

Some MQL Principles: Dollar ($) Notation

Exception: "`$field`" as a value

- `$` indicates that the string is a special keyword
  - E.g., `$gt`, `$lte`, `$add`, `$elemMatch`, ...
- Used as the “field” part of a “field : value” expression
- If binary operator, it is usually written as:
  - `{LOperand : { $keyword : ROperand }}`
  - e.g., `{qty : {$gt : 30}}`
- Alternative: arrays
  - `{$keyword : [argument list]}`
  - e.g., `{$add : [1, 2]}`
- Exception: `$fieldName` on the value side refers to a previously defined field
  - Purpose: disambiguation
  - Only in aggregation pipelines; not now

Some MQL Principles: Dollar ($) Notation

Example

- `$` indicates that the string is a special keyword
  - E.g., `$gt`, `$lte`, `$add`, `$elemMatch`, ...
- Used as the “field” part of a “field : value” expression
- If binary operator, it is usually written as:
  - `{LOperand : { $keyword : ROperand }}`
  - e.g., `{qty : {$gt : 30}}`
- Alternative: arrays
  - `{$keyword : [argument list]}`
  - e.g., `{$add : [1, 2]}`
- Exception: `$fieldName` on the value side refers to a previously defined field
  - Purpose: disambiguation
  - Only in aggregation pipelines; not now

```
{ $project: {
    total: { $multiply: ["$price", "$quantity"]
}
      ↑ operator (key)  ↑ field reference (value)
} }
```

`$multiply` is an **operator**; `"$price"` and `"$quantity"` are **field references**: “use the value of the price and quantity fields”.

Some MQL Principles: Dollar ($) Notation

Why queries are written as documents

MongoDB syntax is quite different from SQL or general-purpose languages because it is a **data-centric DSL** (domain-specific language), designed for expressing data queries and transformations.

Why it looks like this: MongoDB works directly with **data in JSON form**, so instead of writing queries as SQL-style strings it represents them as **JSON objects**.

```
SELECT * FROM orders WHERE qty > 30
```



```
{ qty: { $gt: 30 } }
```

MongoDB Query Language – Retrieval

- Input = collections, output = collections
- Three main types of queries in the query language
  - **Retrieval: Restricted SELECT-WHERE-ORDER BY-LIMIT type queries**
  - Aggregation: A bit of a misnomer; a general pipeline of operators
    - Can capture Retrieval as a special case
    - But worth understanding Retrieval queries first…
  - Updates
- All queries are invoked as
  - `db.collection.operation1(…).operation2(…)…`
    - collection: name of collection
  - Unlike SQL which lists many tables in a FROM clause, MQL is centered around manipulating a single collection

Retrieval Template

db.collection.**find**(*<predicate>*, optional *<projection>*)

- returns documents that match *<predicate>*
- keep fields as specified in *<projection>*
- both *<predicate>* and *<projection>* expressed as documents
- in fact, most things are documents!

Retrieval Template

db.collection.**find**(*<predicate>*, optional *<projection>*)

- returns documents that match *<predicate>*
- keep fields as specified in *<projection>*
- both *<predicate>* and *<projection>* expressed as documents
- in fact, most things are documents!

db.inventory.find({ }) returns all documents

```
db.inventory.find({})
```

DB.INVENTORY.FIND({}) · FIRST THREE OF FIVE


```
{ "_id": ObjectId("5fb5…d993"), "item": "journal", "qty": 25, "size": { "h": 14, "w": 21, "uom": "cm" }, "status": "A", "instock": [ { "loc": "A", "qty": 5 }, { "loc": "C", "qty": 15 } ], "tags": [ "blank", "red" ], "dim": [ 14, 21 ] }
{ "_id": ObjectId("5fb5…d994"), "item": "notebook", "qty": 50, "size": { "h": 8.5, "w": 11, "uom": "in" }, "status": "A", "instock": [ { "loc": "C", "qty": 5 } ], "tags": [ "red", "blank" ], "dim": [ 14, 21 ] }
{ "_id": ObjectId("5fb5…d995"), "item": "paper", "qty": 100, "size": { "h": 8.5, "w": 11, "uom": "in" }, "status": "D", "instock": [ { "loc": "A", "qty": 60 }, { "loc": "B", "qty": 15 } ], "tags": [ "red", "blank", "plain" ], "dim": [ 14, 21 ] }
```


… 2 more documents

Retrieval: Basic Queries

db.collection.find(<predicate>, optional <projection>)

- `find( { status : "D"} )`
  - all documents with status D
- `find ( { qty : {$gte : 50} } )`
  - all documents with qty >= 50
- `find ( { status : "D", qty : {$gte : 50} } )`
  - all documents that satisfy both
- `find( { $or: [ { status : "D" }, { qty : { $lt : 30 } } ] } )`
  - all documents that satisfy either

Retrieval: Nested Documents

db.collection.find(<predicate>, optional <projection>)

- `find( { size: { h: 14, w: 21, uom: "cm" } } )`
    - exact match of nested document, including ordering of fields!
- `find( { "size.uom" : "cm", "size.h" : {$gt : 14 } } )`
    - querying a nested field
    - Note: when using . notation for sub-fields, expression must be in quotes
    - Also note: each argument of find is a document

Retrieval: Arrays

db.collection.find(<predicate>, optional <projection>)

- `find( { tags: ["red", "blank"] } )`
  - Exact match of array
- `find( { tags: "red" } )`
  - Some element matches red
- `find( { tags: { $all: ["red", "plain"] } } )`
  - One element red, another plain
- `find( { dim: { $gt: 15, $lt: 20 } } )`
  - If one element is >15 and another is <20
- `find( { dim: { $elemMatch: { $gt: 15, $lt: 20 } } } )`
  - If a single element is >15 and <20
- `find( { "dim.1": { $gt: 25 } } )`
  - If second item >25; quotes again for . notation

Retrieval: Projection

Keep fields with 1, drop fields with 0

db.collection.find(<predicate>, optional <projection>)

- Use 1s to indicate fields that you want
    - Exception: `_id` is always present unless explicitly excluded
- OR Use 0s to indicate fields you don’t want
- Mixing 0s and 1s is not allowed for non `_id` fields

Retrieval: Projection

Keep fields with 1, drop fields with 0

db.collection.find(<predicate>, optional <projection>)

- Use 1s to indicate fields that you want
  - Exception: `_id` is always present unless explicitly excluded
- OR Use 0s to indicate fields you don't want
- Mixing 0s and 1s is not allowed for non `_id` fields

```
db.inventory.find( { }, { item: 1 } )
```

FIND RESULT

```
{ "_id": ObjectId("5fb5…d993"), "item": "journal" }
{ "_id": ObjectId("5fb5…d994"), "item": "notebook" }
{ "_id": ObjectId("5fb5…d995"), "item": "paper" }
{ "_id": ObjectId("5fb5…d996"), "item": "planner" }
{ "_id": ObjectId("5fb5…d997"), "item": "postcard" }
```

```
db.inventory.find( { }, { item: 1, _id: 0 } )
```

FIND RESULT

```
{ "item": "journal" }
{ "item": "notebook" }
{ "item": "paper" }
{ "item": "planner" }
{ "item": "postcard" }
```

Retrieval: Projection

Keep fields with 1, drop fields with 0

db.collection.find(<predicate>, optional <projection>)

- Use 1s to indicate fields that you want
  - Exception: `_id` is always present unless explicitly excluded
- OR Use 0s to indicate fields you don't want
- Mixing 0s and 1s is not allowed for non `_id` fields

```
db.inventory.find( { }, { item: 1, tags: 0, _id: 0 } )
```

ERROR

```
Cannot do exclusion on field tags in inclusion projection
```

```
db.inventory.find( { }, { item: 1, "instock.loc": 1, _id: 0 } )
```

FIND RESULT

```
{ "item": "journal", "instock": [ { "loc": "A" }, { "loc": "C" } ] }
{ "item": "notebook", "instock": [ { "loc": "C" } ] }
{ "item": "paper", "instock": [ { "loc": "A" }, { "loc": "B" } ] }
{ "item": "planner", "instock": [ { "loc": "A" }, { "loc": "B" } ] }
{ "item": "postcard", "instock": [ { "loc": "B" }, { "loc": "C" } ] }
```

Retrieval: Addendum

Two additional operations that are useful for retrieval:

- Limit (k) like LIMIT in SQL
  - e.g., `db.inventory.find( { } ).limit(1)`
- Sort ({ }) like ORDER BY in SQL
  - List of fields, -1 indicates decreasing 1 indicates ascending

```
db.inventory.find( { }, { _id: 0, instock: 0 } ).sort( { "dim.0": -1, item: 1 } )
```

↑ sort = ORDER BY, -1 = descending

FIND RESULT


```
{ "item": "planner", "qty": 75, "size": { "h": 22.85, "w": 30, "uom": "cm" }, "status": "D", "tags": [ "blank", "red" ], "dim": [ 22.85, 30 ] }
{ "item": "journal", "qty": 25, "size": { "h": 14, "w": 21, "uom": "cm" }, "status": "A", "tags": [ "blank", "red" ], "dim": [ 14, 21 ] }
{ "item": "notebook", "qty": 50, "size": { "h": 8.5, "w": 11, "uom": "in" }, "status": "A", "tags": [ "red", "blank" ], "dim": [ 14, 21 ] }
{ "item": "paper", "qty": 100, "size": { "h": 8.5, "w": 11, "uom": "in" }, "status": "D", "tags": [ "red", "blank", "plain" ], "dim": [ 14, 21 ] }
{ "item": "postcard", "qty": 45, "size": { "h": 10, "w": 15.25, "uom": "cm" }, "status": "A", "tags": [ "blue" ], "dim": [ 10, 15.25 ] }
```

Retrieval: Summary

find, sort and limit as SQL clauses

`find()` = SELECT <projection>
FROM Collection
WHERE <predicate>

`limit()` = LIMIT

`sort()` = ORDER BY

Retrieval: Summary

One query with every part

`find()` = SELECT <projection> FROM Collection WHERE <predicate>

`limit()` = LIMIT

`sort()` = ORDER BY

```
db.inventory.find(
    { tags: "red" },                    // WHERE
    { _id: 0, instock: 0 } )            // SELECT
  .sort( { "dim.0": -1, item: 1 } )     // ORDER BY
  .limit(2)                             // LIMIT
```

```
db.inventory.find({ tags: "red" }, { _id: 0, instock: 0 }).sort({ "dim.0": -1, item: 1 }).limit(2)
```

RESULT

```
{ "item": "planner", "qty": 75, "size": { "h": 22.85, "w": 30, "uom": "cm" }, "status": "D", "tags": [ "blank", "red" ], "dim": [ 22.85, 30 ] }
{ "item": "journal", "qty": 25, "size": { "h": 14, "w": 21, "uom": "cm" }, "status": "A", "tags": [ "blank", "red" ], "dim": [ 14, 21 ] }
```

Recap: The Inventory Collection

```
db.inventory.find({})
```

FIVE DOCUMENTS


```
{ "_id": ObjectId("5fb5…d993"), "item": "journal", "qty": 25, "size": { "h": 14, "w": 21, "uom": "cm" }, "status": "A",
  "instock": [ { "loc": "A", "qty": 5 }, { "loc": "C", "qty": 15 } ], "tags": [ "blank", "red" ], "dim": [ 14, 21 ] }
{ "_id": ObjectId("5fb5…d994"), "item": "notebook", "qty": 50, "size": { "h": 8.5, "w": 11, "uom": "in" }, "status": "A",
  "instock": [ { "loc": "C", "qty": 5 } ], "tags": [ "red", "blank" ], "dim": [ 14, 21 ] }
{ "_id": ObjectId("5fb5…d995"), "item": "paper", "qty": 100, "size": { "h": 8.5, "w": 11, "uom": "in" }, "status": "D",
  "instock": [ { "loc": "A", "qty": 60 }, { "loc": "B", "qty": 15 } ], "tags": [ "red", "blank", "plain" ], "dim": [ 14, 21 ] }
{ "_id": ObjectId("5fb5…d996"), "item": "planner", "qty": 75, "size": { "h": 22.85, "w": 30, "uom": "cm" }, "status": "D",
  "instock": [ { "loc": "A", "qty": 40 }, { "loc": "B", "qty": 5 } ], "tags": [ "blank", "red" ], "dim": [ 22.85, 30 ] }
{ "_id": ObjectId("5fb5…d997"), "item": "postcard", "qty": 45, "size": { "h": 10, "w": 15.25, "uom": "cm" }, "status": "A",
  "instock": [ { "loc": "B", "qty": 15 }, { "loc": "C", "qty": 35 } ], "tags": [ "blue" ], "dim": [ 10, 15.25 ] }
```

Demo: Retrieval

find, projection, sort, limit, and the array trap

1. Items with status `"A"` and qty at least 50: show only `item` and `qty`, no `_id`.
2. Items whose tags include both `"red"` and `"blank"`. Try `{ tags: "red", tags: "blank" }` first: what does it return, and why?
3. Two items with the largest first dimension, ties by item name ascending; show their tags.
4. Items with an `instock` entry at location `"B"` holding at least 30. Check the answer against the documents.

```
// write a query here, then Run   e.g.
db.inventory.find({ status: "D" })
```

RESULT

```
Write a query on the left, then Run.
```

Answer: Retrieval

```
db.inventory.find(
  { "instock.loc": "B", "instock.qty": { $gte: 30 } },
  { _id: 0, item: 1, instock: 1 } )
```

TWO CONDITIONS, TWO DIFFERENT ELEMENTS


```
{ "item": "paper", "instock": [ { "loc": "A", "qty": 60 }, { "loc": "B", "qty": 15 } ] }
{ "item": "planner", "instock": [ { "loc": "A", "qty": 40 }, { "loc": "B", "qty": 5 } ] }
{ "item": "postcard", "instock": [ { "loc": "B", "qty": 15 }, { "loc": "C", "qty": 35 } ] }
```


```
db.inventory.find(
  { instock: { $elemMatch: { loc: "B", qty: { $gte: 30 } } } },
  { _id: 0, item: 1, instock: 1 } )
```

ONE ELEMENT MUST SATISFY BOTH

No documents matched.

All three items with a location-B entry matched on the left, each reaching 30 through a **different element** ({A, 60}, {A, 40}, {C, 35}). `$elemMatch` asks one element to satisfy both, and no item holds 30 at B.

Two conditions on one array: without `$elemMatch` they may be met by **different elements**

Retrieval: Other Features

- Use regexes for matching: `{ item: /^p/ }`
- `$all` : all entries in an array satisfy a condition
- `$in` : checking if a value is present in an array of atomic values
- The presence or absence of fields
  - Can use special “null” values
  - `{field : null}` checks if a field is null or missing
  - `$exists` : checking the presence/absence of a field

Caveat: A Mistyped Field

One document stores qty as the string "50"; the range filter skips it without an error

```
db.inventory.find(
  { qty: { $gte: 30 } },
  { _id: 0, item: 1, qty: 1 } )
```

QTY AT LEAST 30: NOTEBOOK (50) IS MISSING

```
{ "item": "paper", "qty": 100 }
{ "item": "planner", "qty": 75 }
{ "item": "postcard", "qty": 45 }
```

```
db.inventory.find(
  { qty: { $type: "string" } },
  { _id: 0, item: 1, qty: 1 } )
```

WHY: ITS QTY IS A STRING

```
{ "item": "notebook", "qty": "50" }
```

Remedy: validate qty as numeric before accepting the document

This numeric query predicate **skips string values**; numeric BSON types remain comparable

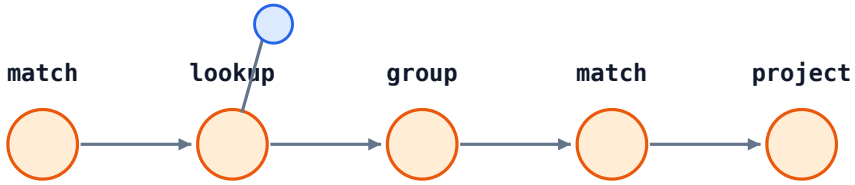
MongoDB Query Language – Aggregation

- Input = collections, output = collections
- Three main types of queries in the query language
  - Retrieval: Restricted SELECT-WHERE-ORDER BY-LIMIT type queries
  - **Aggregation: A bit of a misnomer; a general pipeline of operators**
    - Can capture Retrieval as a special case
    - But worth understanding Retrieval queries first…
  - Updates
- All queries are invoked as
  - `db.collection.operation1(…).operation2(…)…`
    - collection: name of collection
  - Unlike SQL which lists many tables in a FROM clause, MQL is centered around manipulating a single collection

Aggregation Pipelines

A list of stages, run in order

- Composed of a linear *pipeline* of *stages*
- Each stage corresponds to one of:
  - match // first arg of find ( )
  - project // second arg of find ( ) but more expressiveness
  - sort/limit // same
  - group
  - unwind
  - lookup
  - ... lots more!!
- Each stage manipulates the existing collection in some way

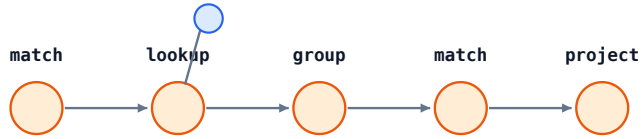


Documents flow left to right; **each stage reads the previous stage's output**

Aggregation Pipelines

A list of stages, run in order

- Composed of a linear *pipeline* of *stages*
- Each stage corresponds to one of:
  - match // first arg of find ( )
  - project // second arg of find ( ) but more expressiveness
  - sort/limit // same
  - group
  - unwind
  - lookup
  - ... lots more!!
- Each stage manipulates the existing collection in some way



- Syntax:
  - ```
    db.collection.aggregate ( [
      { $stage1Op: { } },
      { $stage2Op: { } },
      …
      { $stageNOp: { } }
    ] )
    ```

A pipeline is an **array of stage documents**, executed in the order written
    ```

Grouping Syntax

One output document per `_id`

```
$group : {
    _id: <expression>,        // Group By Expression
    <field1>: { <aggfunc1> : <expression1> },
    … }
```

Returns one document per unique group, indexed by `_id`

Grouping Syntax

Aggregation functions

```
$group : {
    _id: <expression>,        // Group By Expression
    <field1>: { <aggfunc1> : <expression1> },
    … }
```

Returns one document per unique group, indexed by `_id`

Agg. func. can be standard ops like `$sum`, `$avg`, `$max`

- Other useful accumulators:
  - `$first` : return the first expression value per group
    - makes sense only if docs are in a specific order [usually done after sort]
  - `$push` : create an array of expression values per group
    - similar to ARRAY_AGG in SQL systems with array support
  - `$addToSet` : like $push, but eliminates duplicates

The Zips Collection

Twenty zip codes from six US states, in the shape of the MongoDB `zips` sample: `_id` is the zip code, `loc` is [longitude, latitude], `pop` the population.

```
db.zips.find({})
```

DB.ZIPS.FIND({}) · FIRST SIX OF TWENTY

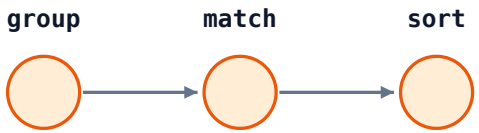

```
{ "_id": "01022", "city": "WESTOVER AFB", "loc": [ -72.559, 42.197 ], "pop": 1764, "state": "MA" }
{ "_id": "01011", "city": "CHESTER", "loc": [ -72.989, 42.279 ], "pop": 1688, "state": "MA" }
{ "_id": "01026", "city": "CUMMINGTON", "loc": [ -72.906, 42.435 ], "pop": 1484, "state": "MA" }
{ "_id": "02115", "city": "BOSTON", "loc": [ -71.097, 42.343 ], "pop": 29178, "state": "MA" }
{ "_id": "90001", "city": "LOS ANGELES", "loc": [ -118.248, 33.974 ], "pop": 57110, "state": "CA" }
{ "_id": "90002", "city": "LOS ANGELES", "loc": [ -118.246, 33.949 ], "pop": 51223, "state": "CA" }
```


… 14 more documents

Grouping: Simple Example

States with at least 200,000 people

Find states with population ≥ 200,000, sorted in descending order



Group first, then filter the groups, then sort: **three stages**

Grouping: Simple Example

States with at least 200,000 people

Find states with population ≥ 200,000, sorted in descending order

```
db.zips.aggregate( [
  { $group: { _id: "$state", totalPop: { $sum: "$pop" } } },
    ↑ GROUP BY   ↑ aggregate
  { $match: { totalPop: { $gte: 200000 } } },
    ↑ after $group = HAVING
  { $sort:  { totalPop: -1 } }  ORDER BY
] )
```

RESULT

```
{ "_id": "NY", "totalPop": 331508 }
{ "_id": "TX", "totalPop": 260264 }
{ "_id": "CA", "totalPop": 217526 }
```

`$group` = GROUP BY + aggregates

`$match` after `$group` = HAVING

Grouping: Simple Example

States with at least 200,000 people

Find states with population ≥ 200,000, sorted in descending order

```
db.zips.aggregate( [
  { $group: { _id: "$state", totalPop: { $sum:
"$pop" } } },
     ↑ GROUP BY
  { $match: { totalPop: { $gte: 200000 } } },
     ↑ after $group = HAVING
  { $sort:  { totalPop: -1 } }
] )
```

RESULT

```
{ "_id": "NY", "totalPop": 331508 }
{ "_id": "TX", "totalPop": 260264 }
{ "_id": "CA", "totalPop": 217526 }
```

Q: what would the SQL query for this be?

```
SELECT   state AS id, SUM(pop) AS
totalPop
FROM     zips
GROUP BY state
HAVING   SUM(pop) >= 200000
ORDER BY totalPop DESC
```

Grouping: Multiple Attributes

City totals, then state averages

```
db.zips.aggregate( [
  { $group: { _id: { state: "$state", city: "$city" }, pop: { $sum: "$pop" } } },
  { $group: { _id: "$_id.state", avgCityPop: { $avg: "$pop" } } }
] )
```

↑ state inside the nested _id of stage 1

Group by 2 attributes, giving a nested `_id`

Group by previously defined `_id.state`

Notice use of `$` to refer to previously defined fields

Q: guesses on what this might be doing?

Two `$group` stages: the second groups **the output of the first**

Grouping: Multiple Attributes

City totals, then state averages

```
db.zips.aggregate( [
  { $group: { _id: { state: "$state", city: "$city" }, pop: { $sum: "$pop" } } },
  { $group: { _id: "$_id.state", avgCityPop: { $avg: "$pop" } } }
] )
```

RESULT

```
{ "_id": "MA", "avgCityPop": 8528.5 }
{ "_id": "CA", "avgCityPop": 72508.667 }
{ "_id": "NY", "avgCityPop": 110502.667 }
{ "_id": "TX", "avgCityPop": 86754.667 }
{ "_id": "WI", "avgCityPop": 43978 }
{ "_id": "IL", "avgCityPop": 192310 }
```



A: find **average city population** per state

Two-level aggregation as **two stages**; SQL would need a subquery

Grouping: Multiple Aggregations

Largest city per state

Find, for every state, the biggest city and its population

```
db.zips.aggregate( [
  { $group: { _id: { state: "$state", city: "$city" },
pop: { $sum: "$pop" } } },
  { $sort:  { pop: -1 } },
  { $group: { _id: "$_id.state", bigCity: { $first:
"$_id.city" }, bigPop: { $first: "$pop" } } },
  { $sort:  { bigPop: -1 } }
] )
```

largest city first

↑ two aggregations per group



Can list multiple aggregations after grouping id

`$sort` then `$first`: the row that **attains the maximum**, per group

Grouping: Multiple Aggregations

Largest city per state

Find, for every state, the biggest city and its population

```
db.zips.aggregate( [
  { $group: { _id: { state: "$state", city: "$city" },
pop: { $sum: "$pop" } } },
  { $sort:  { pop: -1 } },  largest city first
  { $group: { _id: "$_id.state", bigCity: { $first:
"$_id.city" }, bigPop: { $first: "$pop" } } },
      ↑ two aggregations per group
  { $sort:  { bigPop: -1 } }
] )
```

RESULT

```
{ "_id": "IL", "bigCity": "CHICAGO", "bigPop": 192310 }
{ "_id": "NY", "bigCity": "BROOKLYN", "bigPop": 187581 }
{ "_id": "TX", "bigCity": "HOUSTON", "bigPop": 132386 }
{ "_id": "CA", "bigCity": "LOS ANGELES", "bigPop": 108333 }
{ "_id": "WI", "bigCity": "MILWAUKEE", "bigPop": 48094 }
{ "_id": "MA", "bigCity": "BOSTON", "bigPop": 29178 }
```

- Approach:
  - Group by pair of city and state, and compute population per city
  - Order by population descending
  - Group by state, and find first city and population per group (i.e., the highest population city)
  - Order by population descending

Grouping: Projection

Nesting the result

If we wanted to nest the name of the city and population into a nested doc

```
db.zips.aggregate( [
  { $group: { _id: { state: "$state", city: "$city" }, pop:
{ $sum: "$pop" } } },
  { $sort:  { pop: -1 } },
  { $group: { _id: "$_id.state", bigCity: { $first:
"$_id.city" }, bigPop: { $first: "$pop" } } },
  { $sort:  { bigPop: -1 } },
  { $project: { _id: 0, state: "$_id", bigCityDeets: {
name: "$bigCity", pop: "$bigPop" } } }
```

↑ build a nested document

```
] )
```

Can construct new nested documents in output

Grouping: Projection

Nesting the result

If we wanted to nest the name of the city and population into a nested doc

```
db.zips.aggregate( [
  { $group: { _id: { state: "$state", city: "$city" }, pop: { $sum: "$pop" } } },
  { $sort:  { pop: -1 } },
  { $group: { _id: "$_id.state", bigCity: { $first: "$_id.city" }, bigPop: { $first: "$pop" } } },
  { $sort:  { bigPop: -1 } },
  { $project: { _id: 0, state: "$_id", bigCityDeets: { name: "$bigCity", pop: "$bigPop" } } }
    ↑ build a nested document
] )
```

RESULT

```
{ "state": "IL", "bigCityDeets": { "name": "CHICAGO", "pop": 192310 } }
{ "state": "NY", "bigCityDeets": { "name": "BROOKLYN", "pop": 187581 } }
{ "state": "TX", "bigCityDeets": { "name": "HOUSTON", "pop": 132386 } }
{ "state": "CA", "bigCityDeets": { "name": "LOS ANGELES", "pop": 108333 } }
{ "state": "WI", "bigCityDeets": { "name": "MILWAUKEE", "pop": 48094 } }
{ "state": "MA", "bigCityDeets": { "name": "BOSTON", "pop": 29178 } }
```

Demo: Aggregation

$group, $match after $group, $sort then $first

1. Total population per state, only states above 100,000, largest first.
2. How many zip codes does each city in TX have? (one `$match`, one `$group` with `{ $sum: 1 }`)
3. For each state, the city with the most zip codes; break ties by city name. (two groups, a sort, `$first`)

```
// write a query here, then Run   e.g.
db.inventory.find({ status: "D" })
```

RESULT

```
Write a query on the left, then Run.
```

Answer: Aggregation

```
db.zips.aggregate( [
  { $group: { _id: { state: "$state", city:
"$city" }, n: { $sum: 1 } } },
  { $sort:  { n: -1, "_id.state": 1, "_id.city":
1 } },
  { $group: { _id: "$_id.state", city: { $first:
"$_id.city" }, zips: { $first: "$n" } } }
] )
```

RESULT

```
{ "_id": "CA", "city": "LOS ANGELES", "zips": 2 }
{ "_id": "IL", "city": "CHICAGO", "zips": 2 }
{ "_id": "NY", "city": "BROOKLYN", "zips": 2 }
{ "_id": "TX", "city": "HOUSTON", "zips": 2 }
{ "_id": "MA", "city": "BOSTON", "zips": 1 }
{ "_id": "WI", "city": "MADISON", "zips": 1 }
```

`{ $sum: 1 }` counts documents. The second `$group` reads `"$_id.state"`, a field the first one built; `$first` only means “the most” because of the `$sort` before it.

Count with `$sum: 1`; a `$first` is only as meaningful as the `$sort` before it

Unwinding Arrays

Unwind expands an array by constructing documents **one per element** of the array

```
db.inventory.aggregate( [
  { $unwind: "$tags" },   one document per tag
  { $project: { _id: 0, instock: 0, size: 0 } }
] )
```

RESULT · ONE DOCUMENT PER TAG (10 FROM 5)


```
{ "item": "journal", "qty": 25, "status": "A", "tags": "blank",
  "dim": [ 14, 21 ] }
{ "item": "journal", "qty": 25, "status": "A", "tags": "red",
  "dim": [ 14, 21 ] }
{ "item": "notebook", "qty": 50, "status": "A", "tags": "red",
  "dim": [ 14, 21 ] }
{ "item": "notebook", "qty": 50, "status": "A", "tags": "blank",
  "dim": [ 14, 21 ] }
{ "item": "paper", "qty": 100, "status": "D", "tags": "red",
  "dim": [ 14, 21 ] }
{ "item": "paper", "qty": 100, "status": "D", "tags": "blank",
  "dim": [ 14, 21 ] }
{ "item": "paper", "qty": 100, "status": "D", "tags": "plain",
  "dim": [ 14, 21 ] }
{ "item": "planner", "qty": 75, "status": "D", "tags": "blank",
  "dim": [ 22.85, 30 ] }
{ "item": "planner", "qty": 75, "status": "D", "tags": "red",
  "dim": [ 22.85, 30 ] }
{ "item": "postcard", "qty": 45, "status": "A", "tags": "blue",
  "dim": [ 10, 15.25 ] }
```

Unwind: A Common Template

Q: Find sum of qtys across items? A common recipe in MQL queries is to unwind and then group by

```
db.inventory.aggregate( [
  { $unwind: "$instock" },   one document per stock entry
  { $group:  { _id: "$item", totalqty: { $sum:
"$instock.qty" } } }
     ↑ then sum per item
] )
```

RESULT

```
{ "_id": "journal", "totalqty": 20 }
{ "_id": "notebook", "totalqty": 5 }
{ "_id": "paper", "totalqty": 75 }
{ "_id": "planner", "totalqty": 45 }
{ "_id": "postcard", "totalqty": 50 }
```

Looking Up Other Collections

The four fields of `$lookup`

```
{ $lookup: {
    from:         <collection to join>,
    localField:   <referencing field>,
    foreignField: <referenced field>,
    as:           <output array field>
} }
```

- For each document:
  - find documents in other coll that join (equijoin)
    - local field must match foreign field
  - place each of them in an array
- Thus, a left outer equi-join, with the join results stored in an array
- Can do “self-join” to find matching items, including itself, at the same location

Looking Up Other Collections

Example: items sharing a location

```
{ $lookup: {
    from:
<collection to join>,
    localField:
<referencing field>,
    foreignField:
<referenced field>,
    as:           <output
array field>
} }
```

- A left outer equi-join; the matches land in an **array**
- "Self-join": matching items, including itself, at the same location

```
db.inventory.aggregate( [
  { $lookup: { from: "inventory",
               localField: "instock.loc", foreignField:
"instock.loc",
                  ↑ join condition
               as: "otheritems" } },
                  ↑ matches go into an array
  { $project: { _id: 0, item: 1, "otheritems.item": 1 } }
] )
```

RESULT · ITEMS SHARING A LOCATION (FIRST THREE)


```
{ "item": "journal", "otheritems": [ { "item": "journal" }, { "item": "notebook" }, {
  "item": "paper" }, { "item": "planner" }, { "item": "postcard" } ] }
{ "item": "notebook", "otheritems": [ { "item": "journal" }, { "item": "notebook" }, {
  "item": "postcard" } ] }
{ "item": "paper", "otheritems": [ { "item": "journal" }, { "item": "paper" }, { "item":
  "planner" }, { "item": "postcard" } ] }
```


… 2 more documents

Demo: Unwind and Lookup

$unwind then $group; $lookup as a join

1. Total stock of each item across its locations (the slide's template). Then: which stage breaks it if you remove it?
2. Total quantity stocked at each **location**, largest first.
3. For each item, matching items, **including itself**, stocked at a shared location: names only. (Hint: `$lookup` on `"instock.loc"`, then a `$project`.)

```
// write a query here, then Run   e.g.
db.inventory.find({ status: "D" })
```

RESULT

```
Write a query on the left, then Run.
```

Answer: Unwind and Lookup

```
db.inventory.aggregate( [
  { $unwind: "$instock" },
  { $group:  { _id: "$instock.loc",
qty: { $sum: "$instock.qty" } } },
  { $sort:   { qty: -1 } }
] )
```

RESULT

```
{ "_id": "A", "qty": 105 }
{ "_id": "C", "qty": 55 }
{ "_id": "B", "qty": 35 }
```

Unwind puts **one location per document**; only then can `$group` read `"$instock.loc"` as a scalar key. Location A holds the most stock.

The unwind-then-group recipe, grouped by the **array element** this time

Some Rules of Thumb When Writing Queries

- `$project` is helpful to construct or deconstruct nestings (in addition to removing fields or creating new ones)
- `$group` is helpful to construct arrays (using `$push` or `$addToSet`)
- `$unwind` is helpful for unwinding arrays
- `$lookup` is our only hope for joins. Be prepared for a mess. Lots of `$project` needed

MongoDB Query Language – Updates

- Input = collections, output = collections
- Three main types of queries in the query language
  - Retrieval: Restricted SELECT-WHERE-ORDER BY-LIMIT type queries
  - Aggregation: A bit of a misnomer; a general pipeline of operators
    - Can capture Retrieval as a special case
    - But worth understanding Retrieval queries first…
  - **Updates**
- All queries are invoked as
  - `db.collection.operation1(…).operation2(…)…`
    - collection: name of collection
  - Unlike SQL which lists many tables in a FROM clause, MQL is centered around manipulating a single collection

Update Queries: InsertMany

- [Insert/Delete/Update] [One/Many]
  - Many is more general, so we'll discuss that instead

```
db.inventory.insertMany([
  { item: "journal",  qty: 25, size: { h: 14, w: 21, uom: "cm" }, status: "A",
    instock: [ { loc: "A", qty: 5 }, { loc: "C", qty: 15 } ], tags: ["blank", "red"], dim: [14, 21] },
  { item: "notebook", qty: 50, size: { h: 8.5, w: 11, uom: "in" }, status: "A",
    instock: [ { loc: "C", qty: 5 } ], tags: ["red", "blank"], dim: [14, 21] },
  …   // paper, planner, postcard
]);
```

- Several actions will be taken as part of this insert:
  - Will create inventory collection if absent [No schema specification/DDL needed!]
  - Will add the `_id` attribute to each document added (since it isn't there)
  - Will be the first field for each document by default

Update Queries: UpdateMany

Syntax: updateMany ({<condition>}, {<change>})

```
db.inventory.find( { }, { _id: 0, item: 1, dim: 1,
status: 1 } )
```

BEFORE

```
{ "item": "journal", "dim": [ 14, 21 ], "status": "A" }
{ "item": "notebook", "dim": [ 14, 21 ], "status": "A" }
{ "item": "paper", "dim": [ 14, 21 ], "status": "D" }
{ "item": "planner", "dim": [ 22.85, 30 ], "status": "D" }
{ "item": "postcard", "dim": [ 10, 15.25 ], "status": "A" }
```

```
db.inventory.updateMany(
  { "dim.0": { $lt: 15 } },  WHERE
  { $set: { "dim.0": 15, status: "InvalidWidth" } }
  SET
)  // if any width < 15, set it to 15 and set status
to InvalidWidth
db.inventory.find( { }, { _id: 0, item: 1, dim: 1,
status: 1 } )
```

AFTER UPDATEMANY

```
{ "item": "journal", "dim": [ 15, 21 ], "status": "InvalidWidth" }
{ "item": "notebook", "dim": [ 15, 21 ], "status": "InvalidWidth" }
{ "item": "paper", "dim": [ 15, 21 ], "status": "InvalidWidth" }
{ "item": "planner", "dim": [ 22.85, 30 ], "status": "D" }
{ "item": "postcard", "dim": [ 15, 15.25 ], "status": "InvalidWidth"
  }
```

Each document update is **atomic**; use a transaction for all-or-nothing across documents.

Demo: Writes

insertMany creates the collection; updateMany is UPDATE … SET … WHERE

1. Run the insert as given. Look at the `_id` field of the documents afterwards.
2. Every item narrower than 15 gets width 15 and status `"InvalidWidth"` (`updateMany`, `$set`). Then look at the collection.
3. How many documents still have status `"A"`? (`countDocuments`)

```
db.inventory.insertMany([
  { item: "journal", qty: 25,
    status: "A", dim: [14, 21],
    size: { h: 14, w: 21, uom: "cm" },
    tags: ["blank", "red"],
    instock: [ { loc: "A", qty: 5 },
               { loc: "C", qty: 15 } ] },
  { item: "notebook", qty: 50
```

INSERTMANY RESULT


```
{ "acknowledged": true, "insertedCount": 5, "insertedIds":
  [ "65f1c0a1e4b0a1c2d3e40001",
  "65f1c0a1e4b0a1c2d3e40002", "65f1c0a1e4b0a1c2d3e40003",
  "65f1c0a1e4b0a1c2d3e40004", "65f1c0a1e4b0a1c2d3e40005"
  ] }
```


INVENTORY AFTERWARDS


```
{ "_id": ObjectId("65f1…0001"), "item": "journal", "qty":
  25, "status": "A", "dim": [ 14, 21 ], "size": { "h":
  14, "w": 21, "uom": "cm" }, "tags": [ "blank", "red" ],
  "instock": [ { "loc": "A", "qty": 5 }, { "loc": "C",
  "qty": 15 } ] }
```


… 4 more documents

Answer: Writes

```
db.inventory.updateMany( { "dim.0": { $lt: 15 } }, { $set: { "dim.0": 15, status: "InvalidWidth" } } );
db.inventory.countDocuments( { status: "A" } );
db.inventory.find( { }, { item: 1, dim: 1, status: 1 } )
```

AFTER THE INSERT OF THE PREVIOUS SLIDE: UPDATE, THEN LOOK

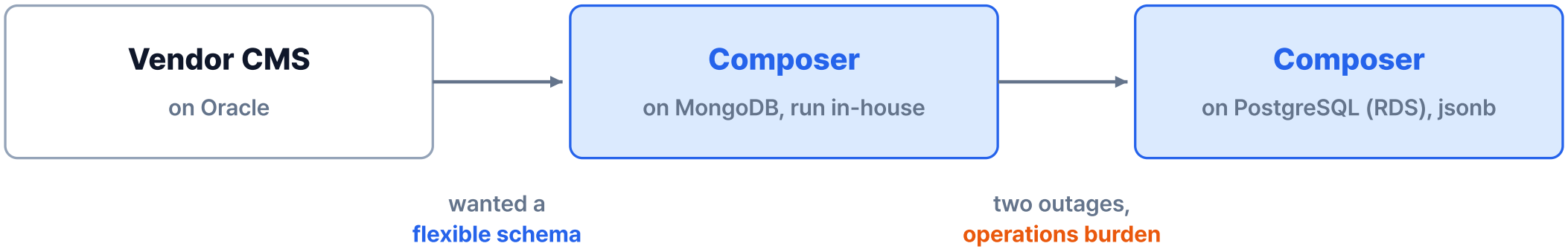
```
{ "_id": ObjectId("5fb5…d993"), "item": "journal", "dim": [ 15, 21 ], "status": "InvalidWidth" }
{ "_id": ObjectId("5fb5…d994"), "item": "notebook", "dim": [ 15, 21 ], "status": "InvalidWidth" }
{ "_id": ObjectId("5fb5…d995"), "item": "paper", "dim": [ 15, 21 ], "status": "InvalidWidth" }
{ "_id": ObjectId("5fb5…d996"), "item": "planner", "dim": [ 22.85, 30 ], "status": "D" }
{ "_id": ObjectId("5fb5…d997"), "item": "postcard", "dim": [ 15, 15.25 ], "status": "InvalidWidth" }
```

The insert created the collection and gave every document an `_id`; the update changed four documents; `countDocuments({ status: "A" })` is now **0**, where it was 3 before.

No DDL, automatic `_id`, and an UPDATE written as **two documents**

Case study: The Guardian's Composer

A newsroom CMS moved off MongoDB in 2018 but kept its documents, as jsonb in PostgreSQL



The documents stayed documents; **the engine** under them changed

Takeaway: MongoDB

1. MongoDB is a **DBMS** with a document model and flexible schemas
2. Compare its validation, transactions, indexes and query plans with **relational systems**
3. Choose the **data model** and the **storage engine** separately